claude-windows / ac48091f-a4f4-4268-b968-75205d9a50ff

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ac48091f-a4f4-4268-b968-75205d9a50ff\subagents\workflows\wf_b8ec0894-3cb\agent-a25aa3ac0a7a44569.jsonl`
- SHA-256: `a720e12b5d5cf4ca092118a77e18b5e7eb035f23ba0d68d821f059144c2fae56`
- Source modified: `2026-06-21T19:14:13+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_b8ec0894-3cb`
- session_id: `ac48091f-a4f4-4268-b968-75205d9a50ff`

Transcript

1. user (2026-06-21T19:12:53.042Z)

```
You are verifying ONE claim in khelsutra's deploy runbook (deploy/DEPLOY_ALPHA.md) against the
REAL code the owner will deploy.

CRITICAL repo rules:
- Engine repo is at /c/Users/avidu/Projects/badminton-highlight-indexer. Its working tree is
checked out on a STALE concurrent-session branch (12 commits behind master). DO NOT read the
working tree for engine claims and DO NOT checkout/commit/modify anything there ? it is a shared
checkout owned by another session.
- To read what the owner actually deploys, read ENGINE origin/master ONLY, via:
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer grep -n "PATTERN" origin/master
-- 'PATHGLOB'      (search master's tree)
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer show origin/master:PATH
(read a master file)
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer log origin/master --oneline -i
--grep="..."        (history)
  (I already ran 'git fetch' so origin/master is current. Never fetch/checkout.)
- khelsutra repo is at /c/Users/avidu/Projects/khelsutra, working tree is on clean master ? you
MAY use Read/Grep/Glob directly there.

Be adversarial and literal: confirm the claim is TRUE for origin/master, with file:line
evidence. If the doc says X but the code says Y, that is a 'discrepancy' ? capture it precisely.
If something the owner must DO (beyond the doc's text) is implied, put it in owner_action_note.
Verdict 'confirmed' only with concrete evidence; 'partial' if mostly true with a caveat;
'refuted' if wrong; 'unknown' if you genuinely could not determine it.

----
CLAIM (§2.2 callout): the boot posture (engine #294) FAILS FAST if edge_auth_enabled is on but
allowed_video_root OR cf_team_domain OR cf_access_aud is missing, and WARNS if PyJWT isn't
installed ? "a half-configured exposure can't silently come up."
On origin/master, find the boot-posture / preflight code (search: `git -C
/c/Users/avidu/Projects/badminton-highlight-indexer grep -n
"edge_auth\|exposure\|preflight\|fail" origin/master -- 'backend/*'`; #294 commit 0c0e064). Read
it via git show. Confirm: (a) it raises/exits when edge_auth_enabled && (allowed_video_root
missing); (b) same for cf_team_domain and cf_access_aud; (c) it only WARNS (not fails) on
missing PyJWT. Quote the ACTUAL error/warning message strings so the owner can recognize them.
Note whether this runs on `--server` startup specifically.
```

2. assistant (2026-06-21T19:12:55.598Z)

```
I'll verify this claim by examining the boot-posture/preflight code in the engine's
origin/master.
```

3. user (2026-06-21T19:12:57.331Z)

origin/master:backend/api/auth.py:3:**DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver is a strict NO-OP (returns
origin/master:backend/api/auth.py:76:   "(pip install -e .[auth]) on a host with edge_auth_enabled."
origin/master:backend/api/auth.py:106:   DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-user; unchanged).
origin/master:backend/api/auth.py:111:   if not sec or not getattr(sec, "edge_auth_enabled", False):
origin/master:backend/api/auth.py:114:   # Strip so a whitespace-only value counts as unset ? matching the boot-time exposure check in
origin/master:backend/api/auth.py:120:   "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not configured")
origin/master:backend/config/models.py:297:   # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local single-user
origin/master:backend/config/models.py:303:   edge_auth_enabled: bool = False
origin/master:backend/config/startup.py:1:"""Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).
origin/master:backend/config/startup.py:18:**Exposure-safety posture (server only, ``include_exposure=True``).** When the engine is *exposed*
origin/master:backend/config/startup.py:19:behind an edge gate (the owner flips ``security.edge_auth_enabled`` on for the Cloudflare-Access
origin/master:backend/config/startup.py:21:A half-configured exposure starts *looking* healthy then leaks / 500s per request, so we verify the
origin/master:backend/config/startup.py:26:``include_exposure=True``; the worker reads ``--video-uri`` (not untrusted tenant paths), so it is
origin/master:backend/config/startup.py:30:a strict **no-op** ? exactly the pre-M5 behaviour; with ``edge_auth_enabled=False`` (today's default)
origin/master:backend/config/startup.py:31:the exposure checks are a strict **no-op** too. So nothing changes for today's manual-knob deployments.
origin/master:backend/config/startup.py:90:def _exposure_checks(config: Any, env: Mapping[str, str]) -> list[StartupCheck]:
origin/master:backend/config/startup.py:91:   """Exposure-safety posture for an *exposed* HTTP server. Gated on ``security.edge_auth_enabled``
origin/master:backend/config/startup.py:94:   rest of the posture at boot so a half-configured exposure fails fast instead of leaking / 500-ing
origin/master:backend/config/startup.py:98:   if not sec or not getattr(sec, "edge_auth_enabled", False):
origin/master:backend/config/startup.py:110:   "security.edge_auth_enabled is ON (engine exposed) but security.allowed_video_root is "
origin/master:backend/config/startup.py:124:   f"security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-Access JWT "
origin/master:backend/config/startup.py:136:   "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ? Cf-Access "
origin/master:backend/config/startup.py:146:   "security.edge_auth_enabled is ON but RALLY_ALLOWED_HOSTS is unset ? the Host-header guard "
origin/master:backend/config/startup.py:159:   include_exposure: bool = False,
origin/master:backend/config/startup.py:165:   ``include_exposure`` adds the exposure-safety posture (``_exposure_checks``) ? the HTTP **server**
origin/master:backend/config/startup.py:167:   DEFAULT-OFF: no profile ? no profile checks; ``edge_auth_enabled=False`` ? no exposure checks.
origin/master:backend/config/startup.py:173:   # Exposure-safety posture ? INDEPENDENT of deployment.profile (a tunnel-to-a-box exposure often
origin/master:backend/config/startup.py:174:   # runs with no profile set), gated on edge_auth_enabled (default-OFF). Server-only.
origin/master:backend/config/startup.py:175:   if include_exposure:
origin/master:backend/config/startup.py:176:   checks.extend(_exposure_checks(config, e))
origin/master:backend/config/startup.py:180:   return checks # no profile ? only the exposure posture (itself off unless edge auth is on).
origin/master:backend/config/startup.py:259:   include_exposure: bool = False,
origin/master:backend/config/startup.py:265:   ``include_exposure`` (server-only) adds the exposure-safety posture. Returns the full findings
origin/master:backend/config/startup.py:270:   config, cuda_available=cuda_available, env=env, include_exposure=include_exposure)
origin/master:backend/main.py:60:from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-OFF)

```
origin/master:backend/main.py:117:    when an edge-auth profile is active
(``security.edge_auth_enabled`` ? i.e. there IS an auth gate
origin/master:backend/main.py:126:    if getattr(sec, "edge_auth_enabled", False):
origin/master:backend/main.py:568:    # unchanged). When edge_auth_enabled, the Cf-Access JWT is
required + verified (a missing or
origin/master:backend/main.py:571:        owner_id = edge_auth.resolve_tenant(request.headers,
global_config)
origin/master:backend/main.py:572:    except edge_auth.EdgeAuthError as e:
origin/master:backend/main.py:848:    the exposed server (``include_exposure=True``, M9
hardening), a half-configured exposure (edge
origin/master:backend/main.py:852:    fail-fast wiring + the default-OFF no-op are unit-
testable."""
origin/master:backend/main.py:854:        enforce_startup_checks(config, logger=logger,
include_exposure=True)
origin/master:backend/main.py:873:                                "security.edge_auth_enabled.
Expose on a network ONLY behind an auth gate.")
origin/master:backend/main.py:940:    _enforce_startup_or_exit(global_config)  # M5: fail fast
on an incoherent ACTIVE profile
origin/master:backend/main.py:1065:            # P2: the friendly launcher ? always localhost-
only (use --server for network exposure),
origin/master:backend/main.py:1085:                      f"gate (edge_auth_enabled / a reverse
proxy) is in front, and set "
origin/master:backend/pipeline/detectors/base.py:147:        A cheap ``test -r`` preflight turns
that into one clear, fixable error BEFORE any
origin/master:backend/pipeline/segmenters/trajectory_hybrid.py:227:        # --- Phase 1: env
healthcheck (fail fast with a clear message) ---
origin/master:backend/utils/redact.py:10:``edge_auth_enabled`` (a boolean), ``cf_access_aud`` (a
public id), or an ML ``max_tokens`` knob.
```

4. user (2026-06-21T19:12:58.174Z)

```
commit 0c0e064071ab5fc2d8c47b5b2f578f0b107c549d
Author: Avi <avi.dullu@gmail.com>
Date:   Sun Jun 21 20:44:48 2026 +0530

    feat(serving): exposure-safety boot posture ? fail fast on a half-configured exposed engine
(default-OFF) (#294)

    M9 hardening for the alpha-shareable "tunnel-to-a-box" topology. When the owner exposes the
    engine behind the Cloudflare-Access gate (flips `security.edge_auth_enabled` on), the gate
is
    only half the safety story ? a half-configured exposure starts looking healthy then leaks /
    500s per request. Verify the rest of the posture at BOOT (server only, via a new
    `include_exposure=True` from `_enforce_startup_or_exit`):

    - EXPOSED_NO_VIDEO_ROOT (fatal): `allowed_video_root` jail unset ? /api/process could read
an
      arbitrary local file.
    - EXPOSED_EDGE_AUTH_INCOMPLETE (fatal): `cf_team_domain`/`cf_access_aud` missing ? lifts the
      current first-request 401 to boot.
    - EXPOSED_AUTH_EXTRA_MISSING (warn): PyJWT[crypto] not importable ? would 500 per request.

    DEFAULT-OFF: `edge_auth_enabled=False` (today's default) ? strict no-op; the worker never
opts
    in (it reads --video-uri, not untrusted tenant paths). Byte-identical to today's
deployments.

    Also aligns `auth.resolve_tenant` to `.strip()` team/AUD so "configured" means the same at
boot
    and at request time (folded from the adversarial review wf_35b735a9, which confirmed
default-OFF
    byte-identical + no bypass). +15 startup tests + a whitespace-config edge-auth test. Tracker
+
    changelog updated in COMPUTE_DECOUPLED_SERVING/README.md.
```

```
    Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

 backend/api/auth.py                       |   6 +-
 backend/config/startup.py                 | 113 ++++++++++++++++++++++----
 backend/main.py                           |  14 ++--
 docs/COMPUTE_DECOUPLED_SERVING/README.md |   3 +-
 tests/test_edge_auth.py                   |   9 +++
 tests/test_startup_checks.py              | 132 ++++++++++++++++++++++++++++++
 6 files changed, 254 insertions(+), 23 deletions(-)
```

5. assistant (2026-06-21T19:13:01.522Z)

Now let me read the full `_exposure_checks` function in startup.py and the surrounding code.

6. user (2026-06-21T19:13:03.211Z)

```
    """True if PyJWT (the ``auth`` extra) is importable ? probed WITHOUT importing it
(find_spec),
    so this stays cheap + side-effect-free. Module-level so tests can monkeypatch it
deterministically
    (whether PyJWT is installed in the test env must not change the WARN's presence)."""
    try:
        import importlib.util
        return importlib.util.find_spec("jwt") is not None
    except Exception:  # pragma: no cover - find_spec only raises on a broken parent package
        return False


def _exposure_checks(config: Any, env: Mapping[str, str]) -> list[StartupCheck]:
    """Exposure-safety posture for an *exposed* HTTP server. Gated on
``security.edge_auth_enabled``
    (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ? the "I am
    exposing this engine behind the CF-Access gate" signal ? the gate alone isn't enough: verify
the
    rest of the posture at boot so a half-configured exposure fails fast instead of leaking /
500-ing
    per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe); ``env`` is the
    caller's environment view (so the host-allowlist check is testable)."""
    sec = getattr(config, "security", None)
    if not sec or not getattr(sec, "edge_auth_enabled", False):
        return []  # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical to
today).

    out: list[StartupCheck] = []

    # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process: WITHOUT
it,
    # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset = "any
local
    # file", the single-user default). Fatal for an exposed engine.
    root = getattr(sec, "allowed_video_root", None)
    if not root or not str(root).strip():
        out.append(StartupCheck(
            LEVEL_ERROR, "EXPOSED_NO_VIDEO_ROOT",
            "security.edge_auth_enabled is ON (engine exposed) but security.allowed_video_root
is "
            "unset ? an authenticated tenant could make /api/process read an arbitrary local
file. "
            "Set allowed_video_root to a jail directory before exposing the engine.",
        ))

    # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only fails
on the
    # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never starts
    # "healthy" then 401s every request.
```

```
    team = (getattr(sec, "cf_team_domain", "") or "").strip()
    aud = (getattr(sec, "cf_access_aud", "") or "").strip()
    if not team or not aud:
        missing = " + ".join(n for n, v in (("cf_team_domain", team), ("cf_access_aud", aud)) if
not v)
        out.append(StartupCheck(
            LEVEL_ERROR, "EXPOSED_EDGE_AUTH_INCOMPLETE",
            f"security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-Access
JWT "
            "cannot be verified (every request would fail). Set security.cf_team_domain "
            "(e.g. https://<team>.cloudflareaccess.com) and security.cf_access_aud (the CF
Access "
            "application AUD tag).",
        ))

    # (3) The auth extra (PyJWT[crypto]) is lazy-imported by auth.verify_cf_access_token. If
it's
    # missing, every request 500s at verify time ? a boot WARN beats a per-request 500. Warning,
not
    # fatal: the auth module's own ImportError is the final authority.
    if not _auth_extra_available():
        out.append(StartupCheck(
            LEVEL_WARN, "EXPOSED_AUTH_EXTRA_MISSING",
            "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ? Cf-Access "
            "verification will fail at request time. Install the 'auth' extra (pip install -e
.[auth]).",
        ))

    # (4) P0c: an exposed engine binds 0.0.0.0, but the Host-header (DNS-rebinding) guard
defaults to
    # loopback-only when RALLY_ALLOWED_HOSTS is unset (see main._allowed_hosts_from_env) ? it
would
    # then 400 every request the edge forwards with its public Host. Boot WARN beats a silent
400-all.
    if not (env.get("RALLY_ALLOWED_HOSTS", "") or "").strip():
        out.append(StartupCheck(
            LEVEL_WARN, "EXPOSED_HOST_ALLOWLIST_LOOPBACK",
            "security.edge_auth_enabled is ON but RALLY_ALLOWED_HOSTS is unset ? the Host-header
guard "
            "defaults to loopback-only and will 400 forwarded requests. Set RALLY_ALLOWED_HOSTS
to your "
            "public host(s) (or '*' when the edge gate validates the host).",
        ))

    return out


def run_startup_checks(
    config: Any,
    *,
    cuda_available: Optional[bool] = None,
    env: Optional[Mapping[str, str]] = None,
    include_exposure: bool = False,
) -> list[StartupCheck]:
    """Return the startup findings for ``config``.

    ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ? GPU checks
    are skipped; the API server passes ``None``, the worker passes
``presets.probe_cuda_available()``).
    ``include_exposure`` adds the exposure-safety posture (``_exposure_checks``) ? the HTTP
**server**
    passes ``True``; the worker leaves it ``False`` (it is never "exposed"). Both dimensions are
    DEFAULT-OFF: no profile ? no profile checks; ``edge_auth_enabled=False`` ? no exposure
checks.
    Pure: no IO (beyond the ``find_spec`` auth probe), no torch import ? the caller owns the GPU
```

```
probe.
    """
    e = os.environ if env is None else env
    checks: list[StartupCheck] = []

    # Exposure-safety posture ? INDEPENDENT of deployment.profile (a tunnel-to-a-box exposure
often
    # runs with no profile set), gated on edge_auth_enabled (default-OFF). Server-only.
    if include_exposure:
        checks.extend(_exposure_checks(config, e))

    profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
    if not profile:
        return checks  # no profile ? only the exposure posture (itself off unless edge auth is
on).

    storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
    indexing = getattr(config, "indexing", None)
    detector_impl = getattr(indexing, "detector_impl", "auto")
    device = getattr(getattr(indexing, "wasb", None), "device", "cuda")

    if profile == "truly-local":
        # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud backend here
is
        # an inconsistent (but not fatal) choice worth surfacing.
        if storage_backend not in ("local", "gdrive"):
            checks.append(StartupCheck(
                LEVEL_WARN, "STORAGE_UNUSUAL",
                f"profile 'truly-local' usually uses local/gdrive storage, but storage.backend="
                f"'{storage_backend}'. Reads will go to a remote backend.",
            ))
        # GPU heads-up (warning only ? the detector healthcheck is the authority; the probe is
        # best-effort and may be False even when a subprocess detector env has CUDA).
        if cuda_available is False:
            if device == "cuda":
                checks.append(StartupCheck(
```

7. assistant (2026-06-21T19:13:06.779Z)

Now let me see how `enforce_startup_checks` handles the ERROR level (raises/exits) and how it's called on server startup. Let me look at the enforcement function and the `_enforce_startup_or_exit` in main.py.

8. user (2026-06-21T19:13:09.008Z)

```
"""Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).

A deployment profile (``truly-local`` / ``cloud-serving`` / ``cpu-mock``) implies a coherent
runtime. These checks **fail/warn fast at process startup** ? a loud signal *before* any
(expensive) job runs ? when the active profile is inconsistent with the environment:

  * **cloud-serving with a non-cloud storage backend** ? a hard **error** (the one config bug we
    can detect with certainty + zero false-positive risk: a cloud profile that can't reach a
    bucket can never serve). This is the README §4.3 "loud error before any job".
  * a **GPU mismatch** (e.g. ``device='cuda'`` on a box where CUDA isn't visible, or
``device='auto'``
    falling back to CPU) ? a **warning**, never an error: the probe is best-effort (torch may
live
    only in the detector subprocess env ? see ``presets.probe_cuda_available``), and the
detector's
    own healthcheck is the real authority. A warning surfaces it early without false-failing.
  * **creds that can't be confirmed** (no ``GOOGLE_APPLICATION_CREDENTIALS`` / not on Cloud Run)
?
    a **warning**: Application Default Credentials can come from the metadata server / gcloud,
which
```

```
    we can't probe offline, so the storage backend's own init is the authority; we just flag it.

**Exposure-safety posture (server only, ``include_exposure=True``).** When the engine is
*exposed*
behind an edge gate (the owner flips ``security.edge_auth_enabled`` on for the Cloudflare-Access
boundary ? the alpha-shareable "tunnel-to-a-box" topology), the gate is only half the safety
story.
A half-configured exposure starts *looking* healthy then leaks / 500s per request, so we verify
the
rest of the posture at **boot**: the ``allowed_video_root`` path-jail is set (else an
authenticated
tenant could read an arbitrary local file ? **error**), the CF team-domain + AUD are present
(else
no token can verify ? **error**, lifting the current first-request failure to boot), and the
``auth`` extra is importable (else every request 500s ? **warning**). Only the HTTP **server**
passes
``include_exposure=True``; the worker reads ``--video-uri`` (not untrusted tenant paths), so it
is
never "exposed" and stays unaffected.

**DEFAULT-OFF (both dimensions):** with no profile selected (``profile == ""``) the profile
checks are
a strict **no-op** ? exactly the pre-M5 behaviour; with ``edge_auth_enabled=False`` (today's
default)
the exposure checks are a strict **no-op** too. So nothing changes for today's manual-knob
deployments.
"""

from __future__ import annotations

import logging
import os
from dataclasses import dataclass
from typing import Any, Mapping, Optional

LEVEL_ERROR = "error"
LEVEL_WARN = "warn"

## Storage backends that count as "a cloud bucket" for cloud-serving coherence.
_CLOUD_STORAGE = ("gcs", "s3")
## STRONG env signals that Google ADC *might* resolve. Deliberately NOT GOOGLE_CL
## CLOUDSDK_CONFIG ? those are often set without usable creds (a plain project id / a
## with no active login), so they would suppress the heads-up falsely. (s3 creds are inten
## left to boto3's default chain ? no parallel heads-up, by design.)
_GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST")


@dataclass(frozen=True)
class StartupCheck:
    """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-up)."""

    level: str
    code: str       # stable, greppable code (e.g. "STORAGE_INCOHERENT")
    message: str


class StartupCheckError(RuntimeError):
    """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL check."""

    def __init__(self, failures: list[StartupCheck]):
        self.failures = failures
```

```
        super().__init__(
            "deployment profile startup check failed: "
            + "; ".join(f"[{c.code}] {c.message}" for c in failures)
        )


def _creds_discoverable(env: Mapping[str, str]) -> bool:
    """True if *some* Google ADC source is hinted by the environment. Best-effort ? a False here
    only downgrades to a WARNING, never a hard failure (the storage init is the authority)."""
    return any(env.get(k) for k in _GCP_ENV_HINTS)


def _auth_extra_available() -> bool:
    """True if PyJWT (the ``auth`` extra) is importable ? probed WITHOUT importing it
(find_spec),
```

9. user (2026-06-21T19:13:09.527Z)

```
    elif profile == "cpu-mock":
        # The mock profile is for CI/regression ? a real detector here defeats the purpose.
        if detector_impl != "stub":
            checks.append(StartupCheck(
                LEVEL_WARN, "MOCK_NONSTUB_DETECTOR",
                f"profile 'cpu-mock' normally uses detector_impl='stub', but detector_impl="
                f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
            ))

    return checks


def enforce_startup_checks(
    config: Any,
    *,
    cuda_available: Optional[bool] = None,
    env: Optional[Mapping[str, str]] = None,
    raise_on_error: bool = True,
    include_exposure: bool = False,
    logger: Optional[logging.Logger] = None,
) -> list[StartupCheck]:
    """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
    ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.

    ``include_exposure`` (server-only) adds the exposure-safety posture. Returns the full
findings
    list (so a caller can inspect/surface them). A no-op with no logging and no raise when both
    dimensions are off (``run_startup_checks`` returns ``[]``)."""
    log = logger or logging.getLogger(__name__)
    checks = run_startup_checks(
        config, cuda_available=cuda_available, env=env, include_exposure=include_exposure)
    for c in checks:
        if c.level == LEVEL_ERROR:
            log.error("[STARTUP] %s: %s", c.code, c.message)
        else:
            log.warning("[STARTUP] %s: %s", c.code, c.message)
    errors = [c for c in checks if c.level == LEVEL_ERROR]
    if errors and raise_on_error:
        raise StartupCheckError(errors)
    return checks
```

10. assistant (2026-06-21T19:13:12.813Z)

Now let me confirm the server-startup wiring in main.py: that `include_exposure=True` is passed only via the server path, and how `_enforce_startup_or_exit` exits.

11. user (2026-06-21T19:13:14.373Z)

```
        print("  [ERROR] No visual frames were extracted in the sample range.")

    print("=" * 60)


def _enforce_startup_or_exit(config) -> None:
    """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL finding
BEFORE
    any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only storage) OR,
for
    the exposed server (``include_exposure=True``, M9 hardening), a half-configured exposure
(edge
    auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when no
    ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU probe
here ?
    the server stays torch-light; the detector healthcheck is the GPU authority. Extracted so
the
    fail-fast wiring + the default-OFF no-op are unit-testable."""
    try:
        enforce_startup_checks(config, logger=logger, include_exposure=True)
    except StartupCheckError as e:
        logger.error("[STARTUP] refusing to start: %s", e)
        sys.exit(1)


def main():
    parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
    parser.add_argument("--output-dir", default="output", help="Directory where processed
clips/highlights are saved")
    parser.add_argument("--setup", action="store_true", help="Launch dependency checker and
setup utility")
    parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video clip
to run detailed diagnostics on")
    parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
    parser.add_argument("--app", action="store_true",
                        help="Launch the app: start the local server (localhost-only) and open
the "
                             "bundled UI in your browser. Falls back to a free port if --port is
busy.")
    parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI server
on")
    parser.add_argument("--host", default=None,
                        help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0 when "
                             "security.edge_auth_enabled. Expose on a network ONLY behind an
auth gate.")
    parser.add_argument("--max-frames", type=int, help="Limit processing to the first N sub-
sampled frames for debugging")

    available_segmenters = list(segmenter_registry.list_available().keys())
    parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
    parser.add_argument("--trim-start", type=float, help="Start time in seconds for the debug
trim segment feature")
    parser.add_argument("--trim-end", type=float, help="End time in seconds for the debug trim
segment feature")
    parser.add_argument("--trim-output", help="Output filename for the debug trimmed segment
(saves in output-dir unless absolute path)")

    args = parser.parse_args()

    if args.setup:
```

```
        # Invoke the diagnostics/bootstrap script (renamed from the old root
        # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
        # build system). Resolve its path from this file so `indexer --setup`
        # works regardless of the caller's cwd.
        import subprocess
        doctor_path = os.path.join(
            os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
            "scripts", "doctor.py",
        )
        print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
        subprocess.run([sys.executable, doctor_path])
        return

    if args.trim_start is not None and args.trim_end is not None:
        if not args.video_path:
            print("[ERROR] Please provide --video-path to extract a segment.")
            return

        # Determine output path
        out_filename = args.trim_output or "trimmed_segment.mp4"
        if os.path.isabs(out_filename):
            out_path = out_filename
            out_dir = os.path.dirname(out_path)
            out_name = os.path.basename(out_path)
        else:
            out_dir = resolve_output_dir(args.output_dir)  # P0a: app-home aware (CWD-identical
when unset)
            out_path = os.path.join(out_dir, out_filename)
            out_name = out_filename

        os.makedirs(out_dir, exist_ok=True)
        print(f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s from
{args.video_path}...")

        # global_config isn't initialized this early; load the typed config so the trim
        # clip carries the same configured stitching paddings + watermark as a real
        # highlight (watermark default-on).
        stitching = load_app_config().stitching
        compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
        try:
            res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
            print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
        except Exception as e:
            print(f"[ERROR] Failed to compile trimmed segment: {e}")
        return

    if args.debug_clip is not None:
        if not args.video_path:
            print("[ERROR] Please provide --video-path to debug a specific clip.")
            return
        run_cli_diagnose_clip(args.video_path, args.debug_clip)
        return

    # Initialize Global Config and DB
    global global_config, db_instance
    global_config = load_app_config()  # returns typed AppConfig
    _enforce_startup_or_exit(global_config)  # M5: fail fast on an incoherent ACTIVE profile

    # The CLI --output-dir overrides the configured output directory. It now lives
    # on the typed model (OutputConfig.output_dir), so every consumer ? including
    # /api/compile ? reads the same value instead of a write-only runtime hack.
    # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
```

```
    # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to before.
    global_config.output.output_dir = resolve_output_dir(args.output_dir)
    os.makedirs(global_config.output.output_dir, exist_ok=True)

    db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)

    # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job sweep
below.
    # The job worker is in-process, so a concurrent process opening this DB would sweep THIS
one's
    # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-released on
exit
    # or crash) is keyed to db_path, so a different --output-dir/DB still runs concurrently.
    global _instance_lock
    try:
        _instance_lock = acquire_lock(db_path + ".lock")
    except OSError as exc:
        # The lock path isn't writable (read-only dir / perms). Don't crash and don't falsely
claim a
```

12. assistant (2026-06-21T19:13:19.464Z)

I have everything I need. Now let me crucially verify that `_enforce_startup_or_exit` (with `include_exposure=True`) runs in the `main()` path that is reached for `--server`. The call at line 940 is in `main()` which runs for both CLI processing and server. Let me confirm there's no early return for `--server` that bypasses it, and check the claim's exact wording in the doc.

13. user (2026-06-21T19:13:21.827Z)

```
        # The lock path isn't writable (read-only dir / perms). Don't crash and don't falsely
claim a
        # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure must not
block a
        # legitimate single instance).
        logger.warning("[JOBS] Could not create the single-instance lock at %s.lock (%s);
proceeding "
                       "WITHOUT the second-instance guard.", db_path, exc)
        _instance_lock = None
    else:
        if _instance_lock is None:
            print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
                  f"start ? a second instance would corrupt in-flight job state. Stop the other
one, or "
                  f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
            sys.exit(1)

    db_instance = Database(db_path)

    # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
    # the executor is in-process. Mark them failed so pollers see a terminal state.
    swept = db_instance.fail_stale_jobs()
    if swept:
        logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

    # CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
single-shot.
    if args.video_path and not args.server and not args.app:
        print(f"[CLI] Processing video file: {args.video_path}")
        # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is fetched to
scratch.
        # The local-existence fast-fail only applies to a local input (stat the RESOLVED key,
not the
        # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error, not a
```

```
crash.
        storage_cfg = getattr(global_config, "storage", None)
        try:
            input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
        except ValueError as e:
            print(f"[FAIL] {e}")
            return
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
            print(f"[FAIL] Video file not found: {args.video_path}")
            return
        local_video = storage.acquire_local_input(args.video_path, storage_cfg)

        video_id = compute_video_id(local_video)
        was_reingest = db_instance.get_video(video_id) is not None

        # Validate (with-context variant surfaces ffprobe_meta for the report)
        print("[CLI] Validating...")
        val_results, val_context = registry.run_all_with_context(local_video, global_config)
        failures = [r for r in val_results if not r.passed]

        # Save video status
        status = "failed" if failures else "processed"
        db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r in
val_results])

        seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
        segmenter_actual = None
        segmentation_ran = False
        try:
            print("\n" + "="*40)
            print("VALIDATION SUMMARY")
            print("="*40)
            for r in val_results:
                status_symbol = "[PASS]" if r.passed else "[FAIL]"
                print(f"{status_symbol} {r.validator_name}: {r.message}")
            print("="*40 + "\n")

            if failures:
                print("[FAIL] Video failed checks. Indexing halted.")
                return

            # Index
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[FAIL] Segmenter '{seg_name}' not found.")
                return

            print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
            segmenter_actual = seg_name
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            segmenter = segmenter_cls(db_instance, global_config)
            result = segmenter.process_video(video_id, local_video, max_frames=args.max_frames)
            segmentation_ran = True

            if isinstance(result, Failure):
                print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
                print("[FAIL] Indexing halted for video.")
                segments = []
            else:
                segments = result.value if hasattr(result, "value") else result
                print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments inside
database: {db_path}")
            # Destroy-AFTER-confirm: keep new on success, restore prior on empty/failure.
            _finalize_reingest(video_id, segments, play_wm, cand_wm)
```

```
        finally:
            # Always emit the per-video observability report. Never raises.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths:
                print(f"[REPORT] Technical report : {paths.get('technical_md')}")
                print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
                print(f"[REPORT] Machine JSON     : {paths.get('json')}")
        return

    # Start Local Server Mode
    if args.app or args.server or not args.video_path:
        if args.app:
            # P2: the friendly launcher ? always localhost-only (use --server for network
exposure),
            # auto-pick a free port if --port is busy, and open the bundled UI once the server
is up.
            bind_host = "127.0.0.1"
            port = _resolve_app_port(bind_host, args.port)
            if port != args.port:
                print(f"[APP] Port {args.port} is busy ? using {port} instead.")
            url = f"http://{bind_host}:{port}/"
            # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback staying
in
            # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-loopback-
only set
            # with --app, or the poll 400s and the browser just won't auto-open ? the server
still runs).
            print(f"[APP] Starting KhelSutra ? opening {url} in your browser "
                  f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop.")
            threading.Thread(target=_open_browser_when_ready, args=(url, url + "api/health"),
                             daemon=True).start()
        else:
            bind_host = _resolve_bind_host(global_config, args.host)  # P0c: config/profile-
driven (default 127.0.0.1)
            port = args.port
            print(f"[SERVER] Starting local FastAPI server on http://{bind_host}:{port}...")
            if bind_host not in ("127.0.0.1", "localhost", "::1"):
                print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ? ensure
an auth "
                      f"gate (edge_auth_enabled / a reverse proxy) is in front, and set "
                      f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header guard
defaults to "
                      f"loopback-only and will 400 forwarded requests otherwise).")
        # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3): the
        # in-process wake-timer for any parked 'waiting' job died with the old process, so
        # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
        _get_executor().submit(_drain_due_jobs)
        uvicorn.run(app, host=bind_host, port=port)

if __name__ == "__main__":
    # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads` resolve
the
    # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ? a
SEPARATE
    # module object. If `main()` runs here it configures `__main__`'s globals, leaving
    # `backend.main.db_instance` = None; the async-job drain then crashes silently on
    # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch through
    # `backend.main` so `main()` configures the module everyone actually reads. (No recursion:
```

14. user (2026-06-21T19:13:21.882Z)

```
1       # Deploy KhelSutra Studio for alpha ? a shareable page, no SSH
2
3       > **Status:** `READY` (owner executes the cloud steps) · **Topology:** split ? SPA on
Cloudflare Pages + engine on an `api.*` cloudflared tunnel, Cloudflare Access over both (the
engine's `HOSTING_PLAN.md` §2 alpha plan). · **Cost:** ~$0/mo standing (Pages + Tunnel + Access
are free ?50 users) + Gemini per-run (~$0.05/match).
4
5       This turns the local-only flow (run engine + SPA on your box, drive it yourself) into a
**page you can share with alpha testers** ? they open `app.khelsutra.guru`, sign in (email
allowlist), point at a video, and get a reel. No SSH, no home IP exposed, no inbound ports.
6
7       ## 0. The shape
8
9       ```
10       tester browser
11          ? https
12          ?
13       Cloudflare edge (free): DNS · TLS · Access (email-OTP allowlist) in front of BOTH
hostnames
14          ??? app.khelsutra.guru  ? Cloudflare Pages         (the web/ SPA; built with
VITE_API_BASE)
15          ??? api.khelsutra.guru  ? cloudflared Tunnel (outbound-only) ?? 127.0.0.1:8000 on
YOUR box
16                                                                          ?  FastAPI engine
17                                                                          ?  (Gemini
segmenter; GPU optional)
18                                                                          ?  async jobs,
SQLite, output/ on disk
19      ```
20
21      - **No inbound ports.** `cloudflared` dials out; the engine stays bound to
`127.0.0.1:8000`. The box's home/public IP is never exposed.
22      - **Auth at the edge.** Cloudflare Access gates both hostnames; the engine *also*
verifies the `Cf-Access` JWT in-app (defence in depth ? `security.edge_auth_enabled`), so a
direct hit to the tunnel can't spoof identity.
23      - **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are different origins ?
the SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64) and the engine
locks CORS to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code change ? it's
already env-driven).
24
25      > **Alternative (single-origin):** if you'd rather run **one box** that serves the SPA
*and* the API behind one origin (Caddy reverse-proxy, no Pages, no cross-origin), see
`LOCAL_APPLIANCE_ARCHITECTURE.md` D1. This runbook ships the **split** topology you chose
(stable Pages-hosted page + a thin API tunnel).
26
27      ---
28
29      ## 1. Prerequisites
30
31      - A **box** to run the engine: the CPU droplet `168.144.126.176` (Gemini-only ? no GPU;
fine for alpha) **or** your Windows+WSL GPU box (real WASB). `ffmpeg` + Python 3.8+ installed;
the `badminton-highlight-indexer` engine checked out.
32      - A **Cloudflare account** with the `khelsutra.guru` zone (you already run the marketing
site there).
33      - `cloudflared` installed on the box (`https://developers.cloudflare.com/cloudflare-
one/connections/connect-networks/downloads/`).
34      - A **fresh** `GEMINI_API_KEY` (rotate the chat-shared one ? it uploads downsized chunks
to Google; this is a paid + cloud action).
35
36      ---
37
38      ## 2. Engine (the `api.` side)
39
```

````
### 2.1 Install (with the auth extra)
```bash
cd badminton-highlight-indexer
pip install -e .[auth]          # PyJWT[crypto] ? REQUIRED when edge auth is on, else
every request 500s
```

### 2.2 The safe-exposed config
Create / edit `config.json` on the box:
```jsonc
{
  "security": {
    "edge_auth_enabled": true,                       // turn the CF-Access JWT verify ON
    "allowed_video_root": "/srv/khelsutra/incoming",  // the path-jail ? REQUIRED when
exposed
    "upload_dir": "/srv/khelsutra/incoming/uploads",  // MUST sit UNDER
allowed_video_root (see §7)
    "cf_team_domain": "https://<your-team>.cloudflareaccess.com",
    "cf_access_aud": "<the CF Access application AUD>"  // filled in §4
  }
}
```
And export the env (a `.env` or your shell):
```bash
export GEMINI_API_KEY="<your-rotated-key>"
export RALLY_CORS_ALLOW_ORIGINS="https://app.khelsutra.guru"   # lock CORS to the SPA
origin
```

> **The engine refuses to start unsafe.** The boot posture (engine PR #294) **fails
fast** if `edge_auth_enabled` is on but the `allowed_video_root` jail or
`cf_team_domain`/`cf_access_aud` is missing, and **warns** if `PyJWT` isn't installed. So a
half-configured exposure can't silently come up ? fix what it prints and restart.

### 2.3 Run
```bash
python -m backend.main --server     # binds 127.0.0.1:8000 (the safe default ? do NOT
change the bind)
```
Keep it alive with a process manager (`systemd` on the droplet, mirroring
`site/scripts/provision-vm.sh`'s unit; or `pm2`/`nssm` on Windows).

---

## 3. The SPA on Cloudflare Pages (the `app.` side)

The SPA ships as a **static Pages site**, git-connected like the marketing site (auto-
deploy on merge to `master`).

1. **Cloudflare dashboard ? Workers & Pages ? Create ? Pages ? Connect to Git** ? pick
the `khelsutra` repo.
2. **Build settings:**
   - **Root directory:** `web`
   - **Build command:** `npm run build`
   - **Build output directory:** `web/dist`
   - **Environment variable:** `VITE_API_BASE = https://api.khelsutra.guru/api`  ? this
is what points the SPA at the engine tunnel (PR #64).
3. **Custom domain:** add `app.khelsutra.guru` to the Pages project.

> Every push to `master` now rebuilds + redeploys the SPA. To preview a change first,
Pages builds a per-PR preview URL.

---

## 4. Cloudflare Access (gate BOTH hostnames)
````

1. **Zero Trust ? Access ? Applications ? Add ? Self-hosted.**
2. **Application domain:** add **both** `app.khelsutra.guru` **and** `api.khelsutra.guru` to the *same* application (one app ? one shared session cookie, so a tester signs in once and both the SPA and its API calls are authorized).
3. **Policy:** Action *Allow*, rule *Emails* ? your alpha testers' addresses (free ? 50 users ? fits 10?25 testers). Use one-time-PIN (email OTP) so testers need no Cloudflare account.
4. **Copy the application `AUD` tag** ? paste into the engine's `config.json` `security.cf_access_aud` (§2.2), and set `cf_team_domain` to `https://<your-team>.cloudflareaccess.com`. Restart the engine.
5. **CORS preflight ?:** the browser sends an unauthenticated `OPTIONS` preflight to `api.` for the cross-origin POSTs. In the Access application's **Settings ? enable ?Bypass options requests to CORS preflight?** (or add an explicit OPTIONS bypass), else the preflight is redirected to the login page and every API call fails. The engine's CORS middleware (`allow_credentials=false`, no cookies) then answers the preflight.

---

5. The tunnel (engine ? Cloudflare)

```bash
cloudflared tunnel login                                    # browser auth to your zone
cloudflared tunnel create khelsutra-studio                  # note the <TUNNEL_UUID>
# copy deploy/cloudflared.config.example.yml ? /etc/cloudflared/config.yml, fill <TUNNEL_UUID>
cloudflared tunnel route dns khelsutra-studio api.khelsutra.guru
cloudflared tunnel run khelsutra-studio                     # or install as a service: `cloudflared service install`
```

See `deploy/cloudflared.config.example.yml` for the ingress (only `api.khelsutra.guru ? http://127.0.0.1:8000`).

---

6. Verify (the posture test + the CUJ)

Exposure posture (must hold ? this is what makes it safe to share):
- From OUTSIDE the box: `curl --max-time 5 http://<box-public-ip>:8000/api/health` ? **refused / timeout** (the engine is `127.0.0.1`-bound; no inbound port). ? The box is not directly reachable.
- `curl -i https://api.khelsutra.guru/api/health` **without** a CF session ? a **302 to the Access login** (not a 200). ? The gate is in front.
- Open `https://app.khelsutra.guru` in a browser ? CF Access login ? after sign-in, the SPA loads and `/api/health` returns 200 (same session cookie). ?

The shareable CUJ (what a tester does): open `app.khelsutra.guru` ? sign in ? paste a `gs://` video (or a path under `allowed_video_root`) ? **Process** ? watch progress ? pick rallies ? **Build** ? **download the reel**. All from the page.

---

7. Honest caveats / deferred

- **`upload_dir` must sit UNDER `allowed_video_root`.** If you enable browser upload, an upload landing outside the jail is rejected by `/api/process` (a functional break, not a hole). Keep `upload_dir` inside the jail (§2.2).
- **You MUST set `edge_auth_enabled=true` to expose.** The boot posture (and the in-app JWT verify) only engage then. Do not put the engine behind the tunnel with edge auth off.
- **Cross-origin reel download:** the reel streams from `api.` with `Content-Disposition: attachment`, so it downloads even cross-origin ? but verify the filename in your browser during the CUJ (cross-origin `<a download>` filename hints are ignored; the header is the authority).
- **Upload size:** Cloudflare's free proxy caps request bodies at **100 MB** ? large matches upload as ?90 MB chunks (the SPA chunked-upload path), or pre-stage to a bucket and ingest the `gs://` URI (already supported).

```
131     - **Box offline = engine offline.** The Pages SPA stays up and shows a friendly "engine
offline" banner (`/api/health` ping); processing resumes when the box is back. Beta removes this
with a queue (`HOSTING_PLAN.md` §3).
132     - **Gemini = paid + uploads footage to Google.** Keep the key rotated; use it only for
clips you're OK sending.
133
134     ---
135
136     ## 8. Teardown
137     `cloudflared tunnel cleanup khelsutra-studio` + delete the tunnel; remove the Pages
custom domain; disable the Access application; stop the engine service. The box's `output/` +
`uploads/` retain originals ? sweep them per your retention policy before any wider exposure.
138
139     ---
140     ### Changelog
141     - 2026-06-21 ? created. Split-topology alpha deploy kit (cloudflared + Pages + CF
Access) consuming the engine exposure-safety boot posture (engine #294) + the SPA
`VITE_API_BASE` (#64). Pairs with `LOCAL_APPLIANCE_ARCHITECTURE.md` (single-origin alternative)
+ the engine `HOSTING_PLAN.md` §2.
142
```

15. assistant (2026-06-21T19:13:33.807Z)

I now have full evidence. The doc claim is at line 65 (the §2.2 callout). Let me cross-check each part of the claim:

Claim parts:
1. "boot posture (engine #294) fails fast if `edge_auth_enabled` is on but `allowed_video_root` OR `cf_team_domain` OR `cf_access_aud` is missing"
2. "warns if `PyJWT` isn't installed"
3. "a half-configured exposure can't silently come up"

Verification against `origin/master`:

(a) `allowed_video_root` missing ? `EXPOSED_NO_VIDEO_ROOT` at `LEVEL_ERROR` (startup.py:106-113). ERROR ? raises `StartupCheckError` (startup.py enforce, ~278) ? `sys.exit(1)` (main.py:855-857). CONFIRMED.

(b) `cf_team_domain` OR `cf_access_aud` missing ? `EXPOSED_EDGE_AUTH_INCOMPLETE` at `LEVEL_ERROR` (startup.py:118-127). The check is `if not team or not aud` ? either missing trips it. CONFIRMED.

(c) PyJWT missing ? `EXPOSED_AUTH_EXTRA_MISSING` at `LEVEL_WARN` (startup.py:131-138), warning only, never raises. CONFIRMED.

Server-startup specificity: `include_exposure=True` is only set in `_enforce_startup_or_exit` (main.py:854), called from `main()` at line 940. This runs for `--server` (and `--app`/no-video-path). Important nuance: the enforcement happens **before** the `--server` branch ? it runs in `main()` for ALL invocations (including CLI `--video-path` processing), not just `--server`. So the exposure check actually runs on every invocation, with `include_exposure=True` always. It's accurate that it runs on `--server` startup, but it's not server-exclusive at the call site (it runs for CLI processing too). The gating that makes it a no-op for non-exposed is `edge_auth_enabled=False`, not the `--server` flag.

One more nuance worth capturing: the worker is the only thing that passes `include_exposure=False`, but `main()` itself always passes `True`. Let me verify the worker path to be precise about the "server only" framing.

16. user (2026-06-21T19:13:35.714Z)

```
origin/master:backend/config/__init__.py:23:    enforce_startup_checks,
origin/master:backend/config/__init__.py:24:    run_startup_checks,
origin/master:backend/config/__init__.py:42:    "enforce_startup_checks",
origin/master:backend/config/__init__.py:43:    "run_startup_checks",
origin/master:backend/config/startup.py:18:**Exposure-safety posture (server only,
```

``include_exposure=True``).** When the engine is *exposed*
origin/master:backend/config/startup.py:26:``include_exposure=True``; the worker reads
``--video-uri`` (not untrusted tenant paths), so it is
origin/master:backend/config/startup.py:63:   """Raised by ``enforce_startup_checks`` when an
active profile fails a FATAL check."""
origin/master:backend/config/startup.py:154:def run_startup_checks(
origin/master:backend/config/startup.py:159:   include_exposure: bool = False,
origin/master:backend/config/startup.py:165:   ``include_exposure`` adds the exposure-safety
posture (``_exposure_checks``) ? the HTTP **server**
origin/master:backend/config/startup.py:175:   if include_exposure:
origin/master:backend/config/startup.py:253:def enforce_startup_checks(
origin/master:backend/config/startup.py:259:   include_exposure: bool = False,
origin/master:backend/config/startup.py:265:   ``include_exposure`` (server-only) adds the
exposure-safety posture. Returns the full findings
origin/master:backend/config/startup.py:267:   dimensions are off (``run_startup_checks``
returns ``[]``)."""
origin/master:backend/config/startup.py:269:   checks = run_startup_checks(
origin/master:backend/config/startup.py:270:   config, cuda_available=cuda_available,
env=env, include_exposure=include_exposure)
origin/master:backend/main.py:53:   enforce_startup_checks,
origin/master:backend/main.py:848:   the exposed server (``include_exposure=True``, M9
hardening), a half-configured exposure (edge
origin/master:backend/main.py:854:   enforce_startup_checks(config, logger=logger,
include_exposure=True)
origin/master:backend/worker/__main__.py:31:   enforce_startup_checks,
origin/master:backend/worker/__main__.py:98:def _run_startup_checks(config: Any) -> bool:
origin/master:backend/worker/__main__.py:111:   enforce_startup_checks(config,
cuda_available=cuda, logger=logger)
origin/master:backend/worker/__main__.py:114:   return False # already logged at ERROR by
enforce_startup_checks
origin/master:backend/worker/__main__.py:160:   if not _run_startup_checks(config):
origin/master:backend/worker/__main__.py:347:   if not _run_startup_checks(config):

17. assistant (2026-06-21T19:13:41.290Z)

Confirmed. The worker (`backend/worker/__main__.py:111`) calls `enforce_startup_checks` WITHOUT `include_exposure` (defaults to False), so the exposure posture is genuinely server/`main()`-only. Only `main.py:854` passes `include_exposure=True`.

All three claim parts are verified against `origin/master`. The claim is fully confirmed. The one minor precision note: the enforcement at `main.py:940` runs in `main()` for every invocation (including CLI `--video-path` processing), not exclusively the `--server` flag ? but since the gate is `edge_auth_enabled` (not the flag) and the deploy always uses `--server`, this doesn't affect correctness of the claim. The doc's "fails fast" / "warns" / "can't silently come up" wording is accurate.

18. user (2026-06-21T19:14:02.180Z)

Structured output provided successfully

19. assistant (2026-06-21T19:14:12.969Z)

Verification complete. The §2.2 callout claim in `C:\Users\avidu\Projects\khelsutra\deploy\DEPLOY_ALPHA.md` (line 65) is **confirmed** against engine `origin/master`.

Key evidence (engine repo `/c/Users/avidu/Projects/badminton-highlight-indexer`, read via `git show origin/master`):
- `backend/config/startup.py:104-127` ? fatal `EXPOSED_NO_VIDEO_ROOT` (missing `allowed_video_root`) and fatal `EXPOSED_EDGE_AUTH_INCOMPLETE` (`if not team or not aud` ? either `cf_team_domain` or `cf_access_aud` missing).
- `backend/config/startup.py:131-138` ? `EXPOSED_AUTH_EXTRA_MISSING` is `LEVEL_WARN` (PyJWT missing ? warn, not fatal).
- `backend/config/startup.py:269-281` + `backend/main.py:854-857` ? ERROR findings raise

`StartupCheckError` ? `sys.exit(1)` with `[STARTUP] refusing to start`; the WARN never raises.
- All gated on `edge_auth_enabled` (default-OFF no-op when false), which §2.2's config.json sets to true.

Two notes for the owner (captured in the structured fields, neither a doc bug):
1. The enforcement runs in `main()` on every invocation with `include_exposure=True`, not literally gated on `--server` ? but it's a no-op unless `edge_auth_enabled=true`, so the §2.3 `--server` boot behaves exactly as the doc says.
2. PyJWT-missing is only a WARN, so the engine boots without `.[auth]` and then 500s every authed request ? §2.1's `pip install -e .[auth]` is what actually prevents that, not the boot posture.